

Software Documentation

GIT

VCS (Version Control System)

VCS tracks every change to your files over time.

Git vs Github

Git

Distributed version control system

Works on local computer

Tracks code changes and versions

Can be used without internet

Created by Linus Torvalds

Github

Cloud based Git hosting system

Works online (cloud)

Stores and shares Git repositories

Usually requires internet for remote collaboration

Owned by Microsoft

First Time Configuration

`git config --global user.name "Your Name"`

ଅବଶ୍ୟକତା ଅନୁଯାୟୀ `commit` ଏହି ନାମ ଦିଅନ୍ତୁ

`git config --global user.email "your@email.com"`
ଅଥବା `commit` ଏହି email ଦିଅନ୍ତୁ (github email use କର better)

`git config --list`
- ଏହା configuration list ଦେଖାଏ

Why we use `--global`?

- ମୁଁ କମ୍ପ୍ୟୁଟରରେ କିଛି Git Repository ତିଆରି କରିବାକୁ ଚାହୁଁ
name ଓ email ସେଟ୍ କରିବାକୁ ହେବ

```
#  
mkdir my-project  
cd my-project  
git init
```

Desktop ଏବଂ ଡିରକ୍ଟରୀ ସମ୍ବନ୍ଧରେ:

Desktop/
└── my-project/

└── .git ← `git init` ତିଆରି କରାଯାଇଛି

Git Clone: Copy existing Repo

```
git clone https://github.com/user/project.git
```

Git Add: staging your changes

```
# git add index.html  
# stage a specific file  
$ git add index.html
```

```
# stage an entire Folder  
$ git add src/
```

```
# stage stage all changes  
$ git add .
```

Example:

```
my-project/  
├──  
├── index.html  
├── style.css  
├── app.js  
├──  
└── src/  
    ├── login.js  
    ├── signup.js  
    └── auth.js
```

Git Commit:

```
$ git commit -m "Add login page"
```

```
$ git commit -m "Fix: null pointer on checkout"
```

बिना इस command add commit करना
जो add वा शामिल रहे

```
$ git commit am -am "Update styles"
```


Inspecting Repository

Git Status

git status

ତଥ୍ୟ :

- Current Branch
- Staged Files
- Untracked Files

Git Log

git log --oneline

↳ commit ସୂଚନା ତଥ୍ୟ line wise

git log --graph --oneline

↳ ସୂଚନା commit history graph ତଥ୍ୟ

Git Diff

જોઈ index.html file change કરેલ,

Before:

```
<h1> Hello </h1>
```

After:

```
<h1> Welcome </h1>
```

\$ git diff

→ Last commit થી આજે current code થી difference

દેખાઈ

Then,

અમર જોઈ કરેલ,

```
$ git add index.html
```

```
$ git diff --staged
```

→ અમર દેખાવે commit કરેલ નહીં પણ જોઈ save કરેલ

Undo Commands

\$ git restore index.html

↳ প্রতি দিনে File আবার আবার commit এ যা হিন,
করে সমস্যা ফিরে চাও

কিন্তু ফেরা আমি stage করে ফেলবো,

মানে git add index.html দিচ্ছো,

গরম রং,

\$ git restore --staged index.html

Stage করা মানে কি?

git add index.html

Commit করার জন্য নির্দিষ্ট পরিবর্তনগুলো নির্বাচন (select)
করে প্রস্তুত রাখা

Advance Undo

Command

safe)

কি undo করে,

git revert

অবচ্ছিন্ন safe

নতুন commit দিচ্ছো undo

git reset --soft

Medium

Commit undo, changes রাখা

git reset --hard

Dangerous

Commit + changes delete

Branching : Git's Superpower

A branch is an independent line of development.

একটি project এর একটি আলাদা development line
আমরা থাকে, যেখানে আমরা নতুন feature তৈরি করা যায়,
কিন্তু main code অক্ষত থাকে না।

1. Isolation: Login Feature বাস্তবায়ন সময় main
website নষ্ট হবে না।

2. Parallel Work:
Developer A → Login
Developer B → Payment
Developer C → Search
আমরা একসাথে কাজ করতে পারি

3. Safety Net: Feature ও Bug ফিরে main
branch নিরাপদ থাকে

Command	ଆବଦ୍ଧ
\$ git branch	List all local branches
\$ git branch feature-login	Create a new branch
\$ git switch feature-login	Switch to a branch
\$ git switch -c feature-login	Create and switch in one tap
\$ git branch -d feature-login	Merge ଥିବାବେଳେ ଏହା delete
\$ git branch -D feature-login	Force delete (Unmerged)

How to Branch merge ?

\$ git switch main

\$ git merge feature-login

\$ git branch -d feature-login

[main branch ଆମର ନିଜସ୍ବ repo - ଏହା ଏକ ଅନ୍ୟ branch].

GitHub Account & SSH

SSH key,

প্রতিবার password না দিয়ে নিম্নলিখিতভাবে github এর সাথে connect করার জন্য]

Push vs Pull

Push

Push মানে Local থেকে Github এ upload,

\$ git push

আমার computer এ থাকা Local commits Github এ upload হয়।

প্রথম Push

\$ git push -u origin main

এরপর থেকে শুরু :

\$ git push

pull

pull মানে github repo অন্য জায়গা হাওয়া updated রকম, সেই latest changes আমার computer এ নিয়ে আসা

\$ git pull

Remote Repositories

ধরে নেওয়া project তোমার computer এ আছে, আর
একই project Github এও আছে।

Local Repository → তোমার computer

Remote Repository → Github এর online copy

Repo connect:

```
$ git remote add origin https://github.com/user/project.git
```

↳ Local project কে Github এর সাথে connect করা

```
$ git push -u origin main
```

↳ এক প্রথম push

```
$ git fetch
```

↳ Github থেকে সুইচ Download করা, merge করা না

```
$ git remote -v
```

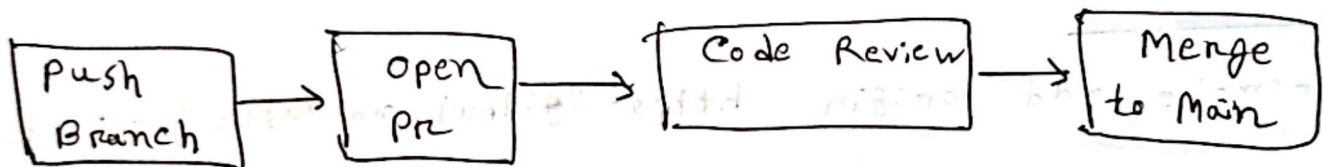
↳ connected remote এর নাম ও url দেখায়

Example:

origin https://github.com/user/project.git

Pull Requests (PR)

A pull request is a proposal to merge your branch into another branch. It's merge process involves code review, discussion and automated checks before merging.



Why we use PR?

- Code Review
- Bug Fix
- Discussion
- Quality Control

Github Workflow

1. create Branch



2. Add commits



3. Open PR



4. Code Review



5. Merge & Deploy

Branching Strategies

1. Git Flow

Bank Mobile App

1. develop থেকে feature/login
2. Login শেষ → develop
3. Release → release / v2.0
4. publish → main
5. সমস্যা Bug → hotfix / payment

2. Github Flow

1. main থেকে feature branch
2. কাজ শেষ
3. pull request
4. Review
5. Merge → main

কোন web app এর জন্য ডাটাবেজ

- কারণ প্রতিদিন হার্ট হার্ট update release করা হয়

3. Trunk-Based Development

Structure:

- ~~এক~~ ~~খ~~ ~~এক~~ branch
- অধিকাংশ অক্ষর main

Best for: Google - এর মতো বড় Team

এখানে Branch অনেকদিন রাখা - হয় না।

একটি Feature → কয়েক ঘণ্টা বা ২ দিনের মধ্যে main-এ merge.

উদ্দেশ্য: অধিকাংশ অক্ষর একই trunk (main) ব্যবহার করে

Why Teams Love Github Flow (slide - ৩২)

- সহজ
- main অক্ষর deployable
- Fast Feedback via PR
- Release branch না রাখা

Trunk - Based Development Features

1. Tiny Commits

- দিনে অনেকবার ছোট commit করা হয়।

2. Feature Flags

- অসম্পূর্ণ feature code-এ থাকে, কিন্তু user দেখতে পায় না।

3. Strong CI/CD

- প্রতিটি commit এর পর automatic test চলে

4. No Long-Lived Branch

- Branch ৩ দিনের বেশি থাকে না।

.gitignore : Keeping Repos Clean

- .gitignore হলো এমন একটি file যেখানে লেখা থাকে কোন কোন file or folder Git Track করতে না।

Example:-

```
node_modules/  
venv/  
.env
```

Rebase vs Merge

Merge: দুই branch এর history combine করে

Advantage:

- History preserve

- Safe

- Shared branch এর জন্য ভাল

Disadvantage:

- Extra merge commit

- History messy হবে

Rebase: নিজের commit নতুন base এর উপর আনা

করা

Advantages:

- Linear History

- Clean git log

Disadvantages:

- Commit hash বদলাবে

- Shared branch এ করা যাবে না

Merge	Release
History Preserve	History rewrite
Merge commit হয়	Merge commit হয় না
Safe	Risky
Team Branch	Personal Branch

git stash:

তুমি login feature নিয়েছো। হঠাৎ boss বসলো
production bug ঠিক করতে। কিন্তু কাজ এখনো
commit না করেছিলি।

Solution → git stash

এটি uncommitted কাজ অস্থায়ীভাবে নুকিয়ে রাখে।

git cherry-pick: Grab a single commit

পুরো branch নয়, শুধু একটি নির্দিষ্ট commit অন্য
branch এ যাবে।